

4.2 결정 트리(Decision Tree)

목차

1. 결정 트리 개요
 1. 결정 트리란
 2. 정보 균일도와 분할 기준
 3. 장단점
2. 결정 트리 모델의 시각화
 1. 붓꽃 데이터 학습
 2. `export_graphviz()`로 트리 파일 생성
 3. Graphviz로 시각화
 4. 피쳐 중요도 확인
3. 결정 트리의 과적합
 1. 실습용 데이터 생성
 2. 결정 경계 시각화 함수
 3. 제약 없는 트리의 결정 경계
 4. `min_samples_leaf` 제약 적용
4. 결정 트리 실습 - Human Activity Recognition
 1. 데이터 소개와 피쳐명 로딩
 2. 중복 피쳐명 확인과 처리
 3. 데이터 로딩 함수
 4. 기본 결정 트리 성능
 5. GridSearchCV로 `max_depth` 튜닝
 6. 테스트 세트에서의 `depth`별 성능
 7. `max_depth`와 `min_samples_split` 동시 튜닝
 8. 최종 모델 성능과 피쳐 중요도
5. 핵심 요약 및 머신러닝 활용 포인트

1. 결정 트리 개요

1.1 결정 트리란

결정 트리(Decision Tree)는 ML 알고리즘 중 직관적으로 이해하기 쉬운 알고리즘입니다. 데이터에 있는 규칙을 학습을 통해 자동으로 찾아내 **트리 기반의 분류 규칙**을 만듭니다. 쉽게 생각하면 **if-else 기반의 규칙**을 자동으로 찾아내 예측을 위한 규칙을 만드는 알고리즘입니다.

구성 요소	설명
루트 노드(Root Node)	트리의 최상위 시작 노드
규칙 노드(Decision Node)	규칙 조건이 되는 노드. 분기가 일어나는 지점
리프 노드(Leaf Node)	더 이상 분기하지 않는 결정된 클래스 값
서브 트리(Sub Tree)	새로운 규칙 조건마다 생성되는 하위 트리

⚠ 규칙이 많을수록 성능이 떨어진다

많은 규칙이 있다는 것은 **분류를 결정하는 방식이 더욱 복잡해진다**는 뜻이고, 이는 곧 **과적합**으로 이어지기 쉽습니다. 즉 **트리의 깊이(depth)가 깊어질수록 예측 성능이 저하될 가능성이 높습니다**.

1.2 정보 균일도와 분할 기준

가능한 한 **적은 결정 노드로 높은 예측 정확도**를 가지려면, 데이터를 분류할 때 **최대한 많은 데이터 세트가 해당 분류에 속할 수 있도록** 결정 노드의 규칙이 정해져야 합니다. 이를 위해 **어떻게 트리를 분할할 것인가**가 중요하며, **최대한 균일한 데이터 세트를 구성할 수 있도록** 분할하는 것이 핵심입니다.

지표	설명	사이킷런
정보 이득 (Information Gain)	엔트로피 개념 기반. 엔트로피는 데이터 집합의 혼잡도 로, 서로 다른 값이 섞여 있으면 높고 같은 값이 섞여 있으면 낮습니다. 정보 이득 = 1 - 엔트로피 이며, 이 값이 큰 속성을 기준으로 분할합니다.	<code>criterion='entropy'</code>
지니 계수 (Gini)	경제학에서 불평등 지수를 나타낼 때 사용하는 계수. 0이 가장 평등하고 1로 갈수록 불평등합니다. ML에서는 지니 계수가 낮을수록 데이터 균일도가 높다고 해석해 낮은 속성을 기준으로 분할합니다.	<code>criterion='gini'</code> (기본값)

1.3 장단점

장점	단점
정보의 균일도 라는 룰을 기반으로 하여 알고리즘이 쉽고 직관적 입니다.	과적합으로 정확도가 떨어집니다. 모든 데이터 상황을 만족하는 완벽한 규칙은 만들 수 없기 때문에, 트리가 복잡해지면서 예측 성능이 저하됩니다. 따라서 트리의 크기를 사전에 제한 하는 튜닝이 오히려 성능 향상에 도움이 됩니다.
규칙 노드와 리프 노드로 시각화 표현이 가능 합니다.	
피처의 스케일링이나 정규화 등 사전 가공의 영향도가 매우 적습니다.	

2. 결정 트리 모델의 시각화

2.1 붓꽃 데이터 학습

```
from sklearn.tree import DecisionTreeClassifier
from sklearn.datasets import load_iris
from sklearn.model_selection import train_test_split
import warnings
warnings.filterwarnings('ignore')

# DecisionTree Classifier 생성
dt_clf = DecisionTreeClassifier(random_state=156)

# 붓꽃 데이터를 로딩하고, 학습과 테스트 데이터 셋으로 분리
iris_data = load_iris()
X_train , X_test , y_train , y_test = train_test_split(iris_data.data, iris_data.target,
                                                    test_size=0.2, random_state=11)

# DecisionTreeClassifier 학습.
dt_clf.fit(X_train , y_train)
```

줄	코드	상세 설명
8	dt_clf = DecisionTreeClassifier(random_state=156)	결정 트리 분류기를 생성합니다. <code>random_state</code> 는 동일한 성능의 분할 후보가 여러 개일 때 선택을 고정해 결과 재현성을 확보합니다.
11~13	load_iris() / train_test_split(...)	붓꽃 데이터 150건을 학습 120건 / 테스트 30건으로 분리합니다.
16	dt_clf.fit(X_train, y_train)	학습을 수행합니다. 이 시점에 트리 구조가 완성되며, 다음 절에서 그 구조를 눈으로 확인합니다.

2.2 export_graphviz()로 트리 파일 생성

```
from sklearn.tree import export_graphviz

# export_graphviz()의 호출 결과로 out_file로 지정된 tree.dot 파일을 생성함.
export_graphviz(dt_clf, out_file="tree.dot", class_names=iris_data.target_names , \
feature_names = iris_data.feature_names, impurity=True, filled=True)
```

줄	코드	상세 설명
1	from sklearn.tree import export_graphviz	사이킷런은 Graphviz 패키지와 쉽게 인터페이스할 수 있는 <code>export_graphviz()</code> API를 제공합니다.
4	export_graphviz(dt_clf, out_file="tree.dot", ...)	학습된 트리를 Graphviz 가 읽을 수 있는 .dot 형식 파일 로 저장합니다. 첫 인자는 학습이 완료된 estimator 여야 합니다.
4	class_names=iris_data.target_names	노드에 표시할 클래스 이름 입니다. 지정하지 않으면 0/1/2 숫자로 표시되어 읽기 어렵습니다.
5	feature_names = iris_data.feature_names	분할 조건에 표시할 피쳐 이름 입니다. <code>X[2] <= 2.45</code> 대신 <code>petal length (cm) <= 2.45</code> 로 표시됩니다.
5	impurity=True	각 노드의 지니 계수 를 표시합니다. 분할이 얼마나 균일도를 높였는지 확인할 수 있습니다.

5	filled=True	노드를 클래스별 색상으로 채웁니다. 색이 진할수록 해당 클래스의 순도가 높다는 뜻입니다.
---	-------------	---

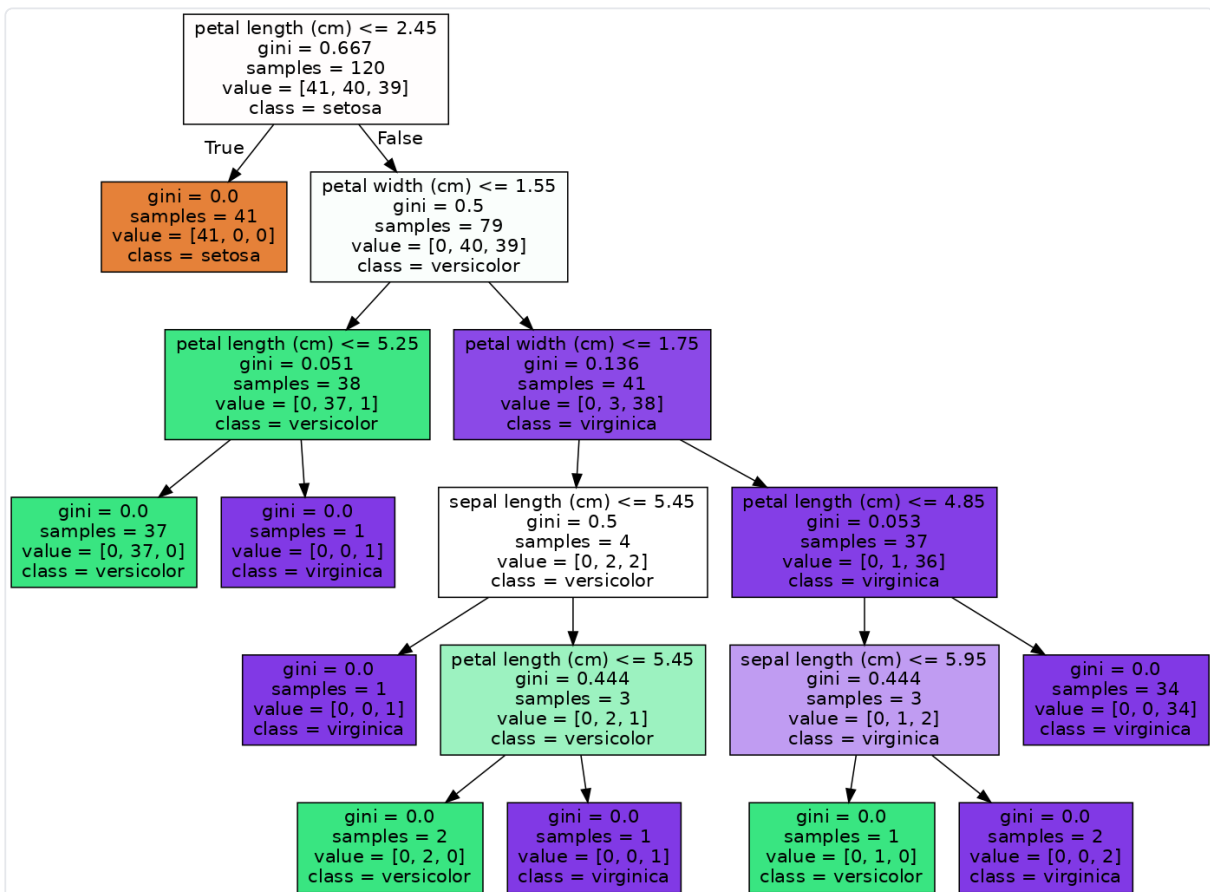
2.3 Graphviz로 시각화

```
import graphviz

# 위에서 생성된 tree.dot 파일을 Graphviz 읽어서 Jupyter Notebook상에서 시각화
with open("tree.dot") as f:
    dot_graph = f.read()
graphviz.Source(dot_graph)
```

줄	코드	상세 설명
4~5	with open("tree.dot") as f: dot_graph = f.read()	<code>with</code> 구문으로 파일을 열어 내용을 문자열로 읽습니다. <code>with</code> 를 쓰면 블록을 벗어날 때 파일이 자동으로 닫힙니다.
6	graphviz.Source(dot_graph)	dot 문자열을 받아 주피터 노트북에 그래프로 렌더링합니다.

▶ 실행 결과



붓꽃 데이터로 학습한 결정 트리의 전체 구조

■ 노드 정보 읽는 법

항목	의미
petal length (cm) <= 2.45	분할 조건. 조건이 True면 왼쪽 , False면 오른쪽 자식으로 이동합니다. 리프 노드에는 이 줄이 없습니다.
gini = 0.663	해당 노드의 지니 계수 . 0에 가까울수록 순수(한 클래스로만 구성)합니다.
samples = 120	해당 노드에 도달한 데이터 건수
value = [41, 40, 39]	클래스별 데이터 분포. 순서는 class_names 와 같습니다.
class = setosa	해당 노드에서 가장 많은 클래스 . 리프 노드라면 이것이 최종 예측값입니다.

루트 노드는 gini 0.663으로 세 클래스가 섞여 있지만, 첫 분할(**petal length <= 2.45**)만으로 **왼쪽 자식이 gini=0.0**인 완전 순수 노드(setosa 41건)가 됩니다. **꽃잎 길이 하나로 setosa가 완벽히 분리된다는** 뜻입니다.

2.4 피쳐 중요도 확인

```
import seaborn as sns
import numpy as np
%matplotlib inline

# feature importance 추출
print("Feature importances:\n{0}".format(np.round(dt_clf.feature_importances_, 3)))

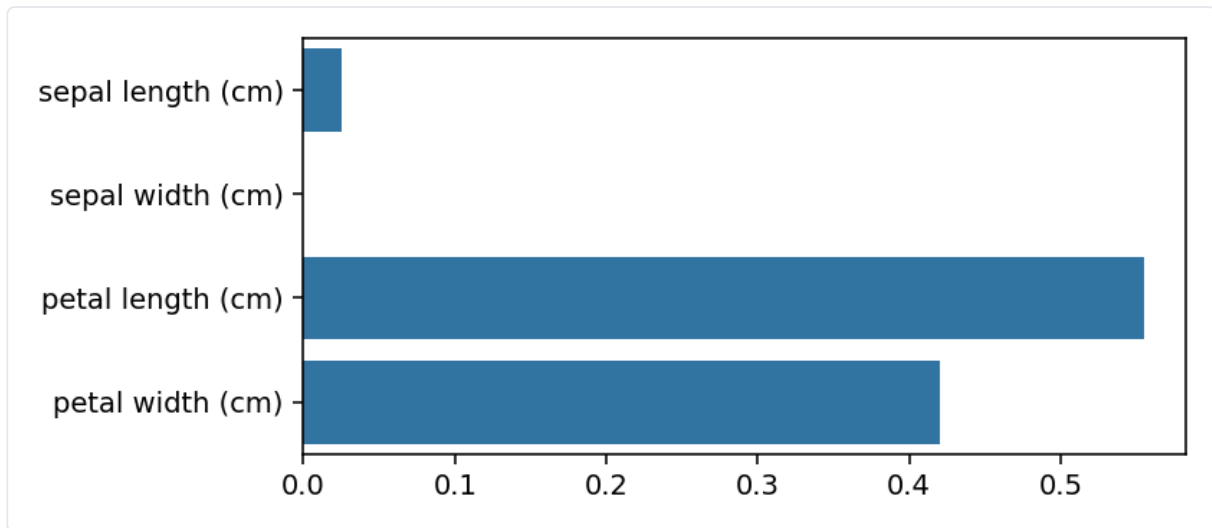
# feature별 importance 매핑
for name, value in zip(iris_data.feature_names , dt_clf.feature_importances_):
    print('{0} : {1:.3f}'.format(name, value))

# feature importance를 column 별로 시각화 하기
sns.barplot(x=dt_clf.feature_importances_ , y=iris_data.feature_names)
```

줄	코드	상세 설명
6	dt_clf.feature_importances_	학습이 끝난 뒤 생성되는 속성으로, 각 피쳐가 규칙을 만드는 데 얼마나 기여했는지 를 나타냅니다. 값이 클수록 중요하며 전체 합은 1 입니다.
9	for name, value in zip(iris_data.feature_names, dt_clf.feature_importances_):	zip() 으로 피쳐 이름과 중요도를 짝지어 순회합니다. 숫자 배열만으로는 어느 피쳐인지 알 수 없기 때문입니다.
13	sns.barplot(x=..., y=iris_data.feature_names)	가로 막대 그래프로 시각화합니다. x에 값, y에 이름을 넣으면 가로 방향이 됩니다.

▶ 실행 결과

```
Feature importances:
[0.025 0.    0.555 0.42 ]
sepal length (cm) : 0.025
sepal width (cm) : 0.000
petal length (cm) : 0.555
petal width (cm) : 0.420
```



붓꽃 4개 피처의 중요도

■ 결과 해석

- **petal length(0.555)와 petal width(0.420)**가 전체 중요도의 **97.5%**를 차지합니다. 붓꽃 품종 분류는 사실상 **꽃잎 정보만으로** 가능하다는 뜻입니다.
- **sepal width는 0.000**으로, 트리에서 **단 한 번도 분할 기준으로 사용되지 않았습니다**. 이런 피처는 제거해도 성능에 영향이 없습니다.
- 피처 중요도는 **피처 선택(feature selection)**의 근거로 활용할 수 있습니다.

3. 결정 트리의 과적합

3.1 실습용 데이터 생성

```
from sklearn.datasets import make_classification
import matplotlib.pyplot as plt
%matplotlib inline

plt.title("3 Class values with 2 Features Sample data creation")

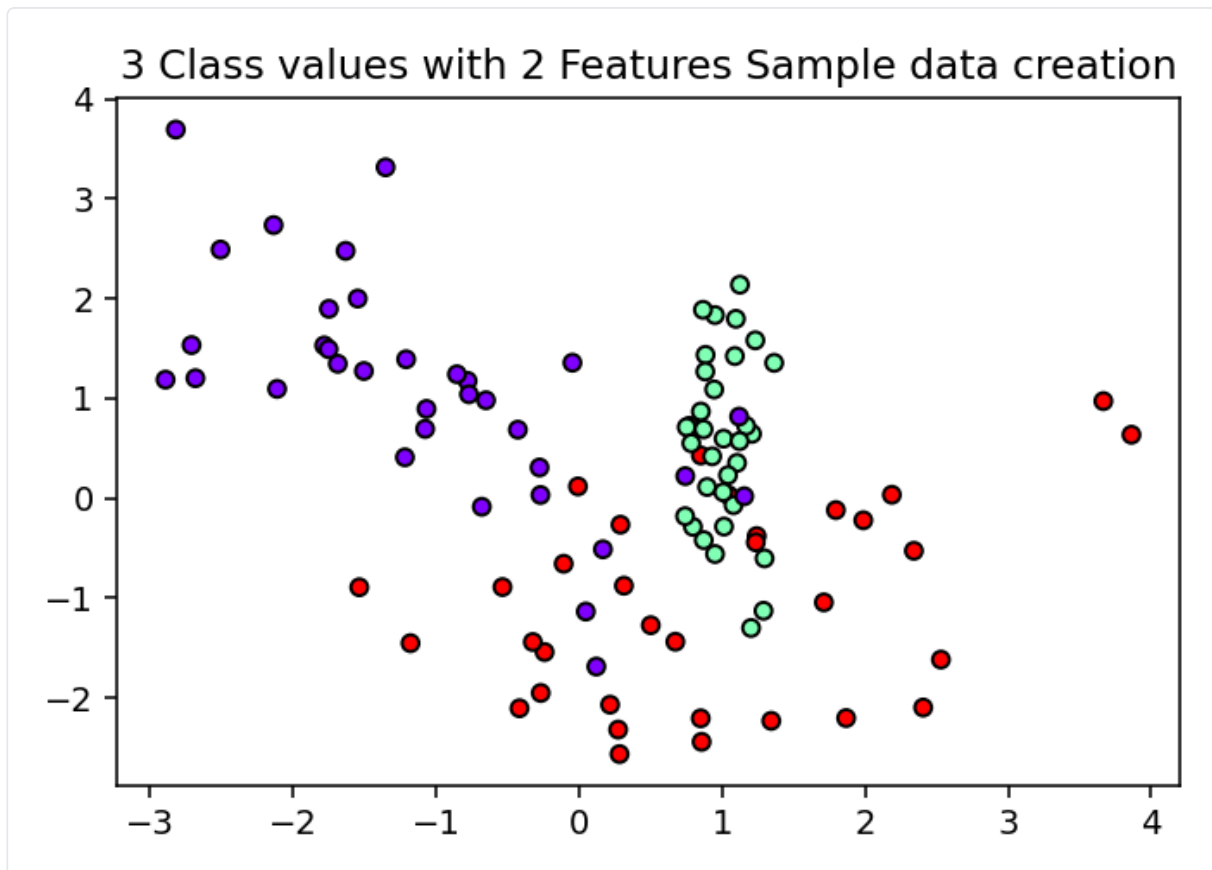
# 2차원 시각화를 위해서 feature는 2개, 결정값 클래스는 3가지 유형의 classification 샘플 데이터 생성.
X_features, y_labels = make_classification(n_features=2, n_redundant=0, n_informative=2,
                                          n_classes=3, n_clusters_per_class=1, random_state=0)

# plot 형태로 2개의 feature로 2차원 좌표 시각화, 각 클래스값은 다른 색깔로 표시됨.
plt.scatter(X_features[:, 0], X_features[:, 1], marker='o', c=y_labels, s=25, cmap='rainbow', edgecolor='k')
```

줄	코드	상세 설명
1	from sklearn.datasets import make_classification	분류용 가상 데이터를 생성하는 함수입니다. 2.3장에서 언급한 <code>make_</code> 계열 API입니다.

8~9	<code>make_classification(n_features=2, n_redundant=0, n_informative=2, n_classes=3, n_clusters_per_class=1, random_state=0)</code>	• <code>n_features=2</code> : 피쳐 2개 → 2차원 평면에 그릴 수 있도록 • <code>n_informative=2</code> : 2개 모두 예측에 유용한 피쳐 • <code>n_redundant=0</code> : 중복 피쳐 없음 • <code>n_classes=3</code> : 클래스 3개 • <code>n_clusters_per_class=1</code> : 클래스당 군집 1개 기본 샘플 수는 100건입니다.
12	<code>plt.scatter(..., c=y_labels, cmap='rainbow', edgecolor='k')</code>	산점도를 그립니다. <code>c=y_labels</code> 로 클래스별 색상을 지정하고, <code>edgecolor='k'</code> 로 점의 테두리를 검게 해 구분을 쉽게 합니다.

▶ 실행 결과



2개 피쳐, 3개 클래스의 가상 데이터

■ 데이터 특성

세 클래스가 대체로 구분되지만, **경계 부근에 다른 클래스로 넘어간 이상치가 몇 개** 보입니다. 이 이상치들이 뒤에서 **과적합을 유발하는 원인**이 됩니다.

3.2 결정 경계 시각화 함수

```
import numpy as np

# Classifier의 Decision Boundary를 시각화 하는 함수
def visualize_boundary(model, X, y):
    fig, ax = plt.subplots()

    # 학습 데이터 scatter plot으로 나타내기
    ax.scatter(X[:, 0], X[:, 1], c=y, s=25, cmap='rainbow', edgecolor='k',
```



```

        clim=(y.min(), y.max()), zorder=3)
ax.axis('tight')
ax.axis('off')
xlim_start , xlim_end = ax.get_xlim()
ylim_start , ylim_end = ax.get_ylim()

# 호출 파라미터로 들어온 training 데이터로 model 학습 .
model.fit(X, y)
# meshgrid 형태인 모든 좌표값으로 예측 수행.
xx, yy = np.meshgrid(np.linspace(xlim_start,xlim_end, num=200),np.linspace(ylim_start,ylim_end, num=200))
Z = model.predict(np.c_[xx.ravel(), yy.ravel()]).reshape(xx.shape)

# contourf() 를 이용하여 class boundary 를 visualization 수행.
n_classes = len(np.unique(y))
contours = ax.contourf(xx, yy, Z, alpha=0.3,
                      levels=np.arange(n_classes + 1) - 0.5,
                      cmap='rainbow', clim=(y.min(), y.max()),
                      zorder=1)

```

줄	코드	상세 설명
4	def visualize_boundary(model, X, y):	모델의 결정 경계(decision boundary) 를 그리는 함수입니다. <code>model</code> 을 인자로 받으므로 어떤 분류기든 재사용 가능합니다.
8~9	ax.scatter(..., zorder=3)	학습 데이터를 산점도로 그림니다. <code>zorder=3</code> 은 그리기 순서 로, 값이 클수록 위에 그려집니다. 점이 경계 색 위에 오도록 합니다.
10~11	ax.axis('tight') / ax.axis('off')	축 범위를 데이터에 딱 맞추고 , 축 눈금과 테두리를 숨깁니다 .
12~13	xlim_start, xlim_end = ax.get_xlim()	현재 그래프의 x, y 범위 를 가져옵니다. 이 범위 안을 격자로 채워 예측할 것입니다.
16	model.fit(X, y)	함수 안에서 학습 을 수행합니다.
18	xx, yy = np.meshgrid(np.linspace(...,200), np.linspace(...,200))	핵심 아이디어 입니다. <code>linspace</code> 로 각 축을 200등분하고 <code>meshgrid</code> 로 200×200 = 40,000개의 격자 좌표 를 만듭니다. 평면 전체를 촘촘히 덮는 가상의 점들입니다.
19	Z = model.predict(np.c_[xx.ravel(), yy.ravel()]).reshape(xx.shape)	<code>ravel()</code> : 2차원 격자를 1차원으로 펼침 <code>np.c_[...]</code> : 두 배열을 열 방향으로 결합해 (40000, 2) 형태로 만들 <code>predict</code> : 40,000개 점을 모두 예측 <code>reshape(xx.shape)</code> : 결과를 다시 200×200 격자 형태로 복원 이렇게 하면 평면의 모든 위치에서 모델이 어떤 클래스로 예측하는지 알 수 있습니다.
23~27	ax.contourf(xx, yy, Z, alpha=0.3, levels=..., zorder=1)	등고선 채우기 로 영역을 색칠합니다. <code>alpha=0.3</code> 으로 반투명하게 해 데이터 점이 보이게 하고, <code>levels</code> 로 클래스 경계를 정확히 맞춥니다.

3.3 제약 없는 트리의 결정 경계

```

from sklearn.tree import DecisionTreeClassifier

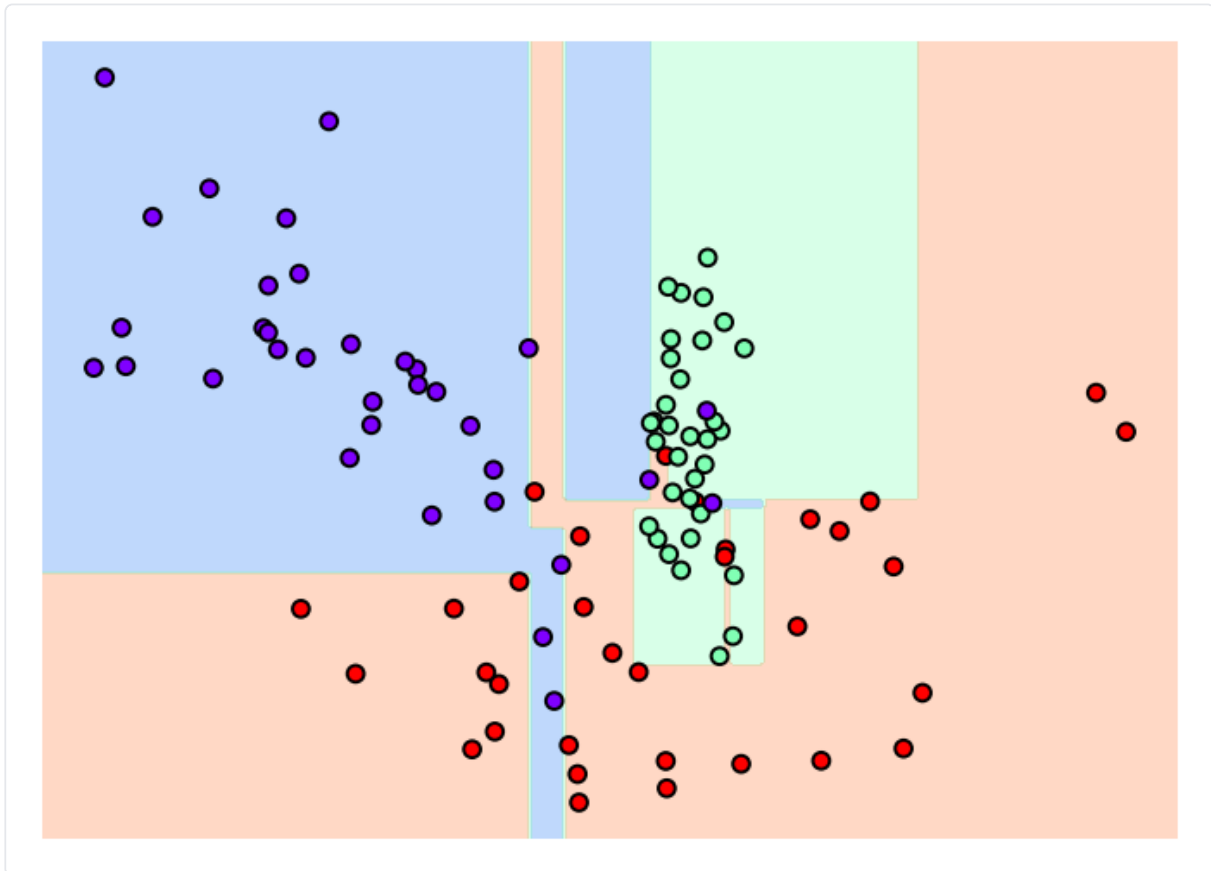
# 특정한 트리 생성 제약없는 결정 트리의 Decsion Boundary 시각화.
dt_clf = DecisionTreeClassifier(random_state=156).fit(X_features, y_labels)
visualize_boundary(dt_clf, X_features, y_labels)

```

줄	코드	상세 설명
---	----	-------

4	<code>DecisionTreeClassifier(random_state=156).fit(X_features, y_labels)</code>	아무런 제약 조건 없이 결정 트리를 학습합니다. 기본값 <code>max_depth=None</code> 이므로 모든 데이터를 완벽히 분류할 때까지 트리를 무한정 확장합니다.
---	---	---

▶ 실행 결과



제약 없는 결정 트리의 결정 경계

⚠ 결과 해석 — 전형적인 과적합

결정 경계가 매우 복잡하고 들쭉날쭉합니다. 특히 다른 클래스 영역에 홀로 떨어져 있는 이상치 하나까지 정확히 분류하기 위해 얇고 뾰족한 영역이 만들어졌습니다.

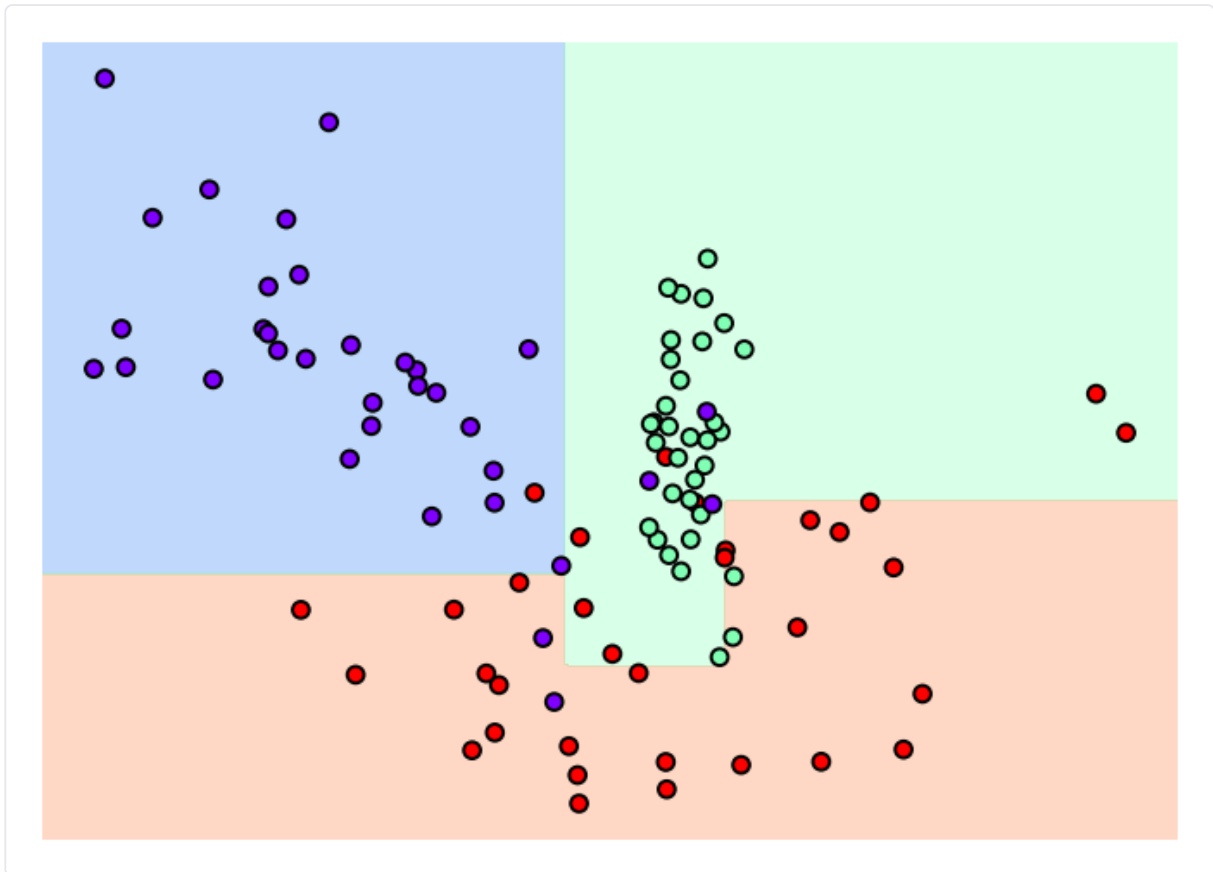
이렇게 학습된 모델은 학습 데이터는 100% 맞히지만, 조금만 다른 새 데이터가 들어오면 정확도가 크게 떨어집니다. 이것이 결정 트리의 대표적인 약점인 과적합입니다.

3.4 min_samples_leaf 제약 적용

```
# min_samples_leaf=6 으로 트리 생성 조건을 제약한 Decision Boundary 시각화
dt_clf = DecisionTreeClassifier(min_samples_leaf=6, random_state=156).fit(X_features, y_labels)
visualize_boundary(dt_clf, X_features, y_labels)
```

줄	코드	상세 설명
2	<code>DecisionTreeClassifier(min_samples_leaf=6, random_state=156)</code>	<code>min_samples_leaf=6</code> 은 리프 노드가 되기 위해 최소 6개의 샘플이 필요하다는 제약입니다. 샘플이 6개 미만이면 더 이상 분할하지 않으므로, 이상치 하나를 위한 분할이 원천적으로 차단됩니다.

▶ 실행 결과



min_samples_leaf=6 제약을 준 결정 트리의 결정 경계

■ 결과 해석 — 제약이 일반화 성능을 높인다

결정 경계가 **훨씬 단순하고 매끄러워졌습니다**. 이상치 몇 개는 잘못 분류하지만, 데이터의 **전반적인 패턴을 훨씬 잘 반영합니다**. 학습 데이터 정확도는 떨어졌지만 **새로운 데이터에 대한 예측 성능(일반화 성능)**은 오히려 향상됩니다. 이것이 하이퍼 파라미터로 트리를 제약하는 이유입니다.

■ 결정 트리의 주요 규제 파라미터

파라미터	설명	과적합 대응
max_depth	트리의 최대 깊이	낮춘다
min_samples_split	노드를 분할하기 위한 최소 샘플 수 (기본 2)	높인다
min_samples_leaf	리프 노드가 되기 위한 최소 샘플 수 (기본 1)	높인다
max_features	분할에 고려할 최대 피쳐 수	제한한다
max_leaf_nodes	리프 노드의 최대 개수	제한한다

4. 결정 트리 실습 - Human Activity Recognition

4.1 데이터 소개와 피처명 로딩

UCI Human Activity Recognition 데이터 세트는 30명의 실험자가 스마트폰을 착용한 채 수행한 동작을 **가속도계·자이로스코프 센서**로 측정한 데이터입니다. **561개의 피처로 6가지 동작**(걷기, 계단 오르기/내리기, 앉기, 서기, 눕기)을 예측합니다.

```
import pandas as pd
import matplotlib.pyplot as plt
%matplotlib inline

# features.txt 파일에는 피처 이름 index와 피처명이 공백으로 분리되어 있음. 이를 DataFrame으로 로드.
feature_name_df = pd.read_csv('./human_activity/features.txt', sep=' ',
                              header=None, names=['column_index', 'column_name'])

# 피처명 index를 제거하고, 피처명만 리스트 객체로 생성한 뒤 샘플로 10개만 추출
feature_name = feature_name_df.iloc[:, 1].values.tolist()
print('전체 피처명에서 10개만 추출:', feature_name[:10])
```

줄	코드	상세 설명
6~7	pd.read_csv('...features.txt', sep=' ', header=None, names=[...])	- sep=' ' : 구분자가 콤마가 아니라 하나 이상의 공백이라는 정규식 - header=None : 파일에 컬럼명 줄이 없음 - names=[...] : 컬럼명을 직접 부여
10	feature_name = feature_name_df.iloc[:, 1].values.tolist()	두 번째 컬럼(피처명)만 뽑아 DataFrame → ndarray → 리스트 로 변환합니다. read_csv의 names 인자는 리스트를 요구하기 때문입니다.

▶ 실행 결과

```
전체 피처명에서 10개만 추출: ['tBodyAcc-mean()-X', 'tBodyAcc-mean()-Y', 'tBodyAcc-mean()-Z',
'tBodyAcc-std()-X', 'tBodyAcc-std()-Y', 'tBodyAcc-std()-Z', 'tBodyAcc-mad()-X',
'tBodyAcc-mad()-Y', 'tBodyAcc-mad()-Z', 'tBodyAcc-max()-X']
```

■ 피처명 규칙

t 로 시작하면 **시간 영역(time)**, f 로 시작하면 **주파수 영역(frequency)** 신호입니다. BodyAcc 는 신체 가속도, GravityAcc 는 중력 가속도, Gyro 는 자이로스코프를 의미하며, 끝의 -X/-Y/-Z 는 축을 나타냅니다.

4.2 중복 피처명 확인과 처리

```
feature_dup_df = feature_name_df.groupby('column_name').count()
print(feature_dup_df[feature_dup_df['column_index'] > 1].count())
feature_dup_df[feature_dup_df['column_index'] > 1].head()
```

줄	코드	상세 설명
1	feature_name_df.groupby('column_name').count()	피처명으로 그룹화해 각 이름이 몇 번 등장하는지 셉니다.
2	feature_dup_df[feature_dup_df['column_index'] > 1].count()	등장 횟수가 1보다 큰(=중복된) 피처명의 개수를 출력합니다. 42개가 중복 입니다.

▶ 실행 결과

```
column_index    42
dtype: int64
```

```
column_index
column_name
fBodyAcc-bandsEnergy()-1,16    3
fBodyAcc-bandsEnergy()-1,24    3
fBodyAcc-bandsEnergy()-1,8     3
fBodyAcc-bandsEnergy()-17,24   3
fBodyAcc-bandsEnergy()-17,32   3
```

⚠ 중복 피처명이 일으키는 문제

`bandsEnergy` 계열 피처명이 각각 3번씩 나타납니다. 이 상태로 DataFrame의 컬럼명으로 사용하면 신규 버전 판다스에서 "Duplicate names" 오류가 발생합니다. 따라서 중복된 이름에 순번을 붙여 고유하게 만들어야 합니다.

```
def get_new_feature_name_df(old_feature_name_df):
    feature_dup_df = pd.DataFrame(data=old_feature_name_df.groupby('column_name').cumcount(),
                                  columns=['dup_cnt'])
    feature_dup_df = feature_dup_df.reset_index()
    new_feature_name_df = pd.merge(old_feature_name_df.reset_index(), feature_dup_df, how='outer')
    new_feature_name_df['column_name'] = new_feature_name_df[['column_name', 'dup_cnt']].apply(
        lambda x : x[0]+'_'+str(x[1]) if x[1] > 0 else x[0] , axis=1)
    new_feature_name_df = new_feature_name_df.drop(['index'], axis=1)
    return new_feature_name_df
```

줄	코드	상세 설명
2~3	<code>old_feature_name_df.groupby('column_name').cumcount()</code>	핵심 함수입니다. <code>cumcount()</code> 는 그룹 내에서 0부터 시작하는 누적 순번을 매깁니다. 같은 이름이 3번 나오면 0, 1, 2가 부여됩니다.
4	<code>feature_dup_df.reset_index()</code>	인덱스를 컬럼으로 꺼내 병합의 기준으로 삼습니다.
5	<code>pd.merge(old_feature_name_df.reset_index(), feature_dup_df, how='outer')</code>	원본 피처명 표와 순번 표를 인덱스 기준으로 병합합니다. <code>how='outer'</code> 는 양 쪽 모두 유지합니다.
6~7	<code>apply(lambda x : x[0]+'_'+str(x[1]) if x[1]>0 else x[0], axis=1)</code>	순번이 0보다 클 때만 이름 뒤에 _순번 을 붙입니다. 첫 번째 등장(순번 0)은 원래 이름 그대로 유지됩니다. <code>axis=1</code> 은 행 단위로 함수를 적용하라는 뜻입니다.
8	<code>drop(['index'], axis=1)</code>	병합용으로 만든 임시 index 컬럼을 삭제합니다.

⚠ pandas 2.x 호환 주의

lambda 안의 `x[0]` , `x[1]` 은 위치 기반 접근인데, pandas 2.x에서는 Series에 정수 인덱싱을 하면 `FutureWarning`이 발생하거나 향후 동작이 달라질 수 있습니다. 안정화 버전에서는 이름으로 접근하는 것이 안전합니다.

```
.apply(lambda x: x['column_name']+'_'+str(x['dup_cnt']) if x['dup_cnt']>0
        else x['column_name'], axis=1)
```

4.3 데이터 로딩 함수

```
import pandas as pd

def get_human_dataset( ):
```

```
# 각 데이터 파일들은 공백으로 분리되어 있으므로 read_csv에서 공백 문자를 sep으로 할당.
feature_name_df = pd.read_csv('./human_activity/features.txt', sep='\\s+',
                              header=None, names=['column_index', 'column_name'])

# 중복된 피처명을 수정하는 get_new_feature_name_df()를 이용, 신규 피처명 DataFrame생성.
new_feature_name_df = get_new_feature_name_df(feature_name_df)

# DataFrame에 피처명을 컬럼으로 부여하기 위해 리스트 객체로 다시 변환
feature_name = new_feature_name_df.iloc[:, 1].values.tolist()

# 학습 피쳐 데이터 셋과 테스트 피쳐 데이터를 DataFrame으로 로딩. 컬럼명은 feature_name 적용
X_train = pd.read_csv('./human_activity/train/X_train.txt', sep='\\s+', names=feature_name)
X_test = pd.read_csv('./human_activity/test/X_test.txt', sep='\\s+', names=feature_name)

# 학습 레이블과 테스트 레이블 데이터를 DataFrame으로 로딩하고 컬럼명은 action으로 부여
y_train = pd.read_csv('./human_activity/train/y_train.txt', sep='\\s+', header=None, names=['action'])
y_test = pd.read_csv('./human_activity/test/y_test.txt', sep='\\s+', header=None, names=['action'])

# 로드된 학습/테스트용 DataFrame을 모두 반환
return X_train, X_test, y_train, y_test
```

```
X_train, X_test, y_train, y_test = get_human_dataset()
```

줄	코드	상세 설명
3	def get_human_dataset():	데이터 로딩 전 과정을 하나의 함수로 묶습니다 . 4.4장, 4.5장에서도 그대로 재사용됩니다.
9	new_feature_name_df = get_new_feature_name_df(feature_name_df)	앞에서 만든 중복 제거 함수 를 호출해 고유한 피처명을 확보합니다.
15~16	X_train = pd.read_csv('...X_train.txt', sep='\\s+', names=feature_name)	피쳐 데이터를 로딩하며 names 로 561개 컬럼명을 부여 합니다. 파일 자체에는 컬럼명이 없습니다.
19~20	y_train = pd.read_csv('...y_train.txt', ..., names=['action'])	레이블을 로딩하고 컬럼명을 action 으로 지정합니다. 값은 1~6의 동작 코드 입니다.
26	X_train, X_test, y_train, y_test = get_human_dataset()	함수가 4개 값을 튜플로 반환 하므로 한 줄에 연패킹합니다.

```
print('## 학습 피쳐 데이터셋 info()')
print(X_train.info())
```

▶ 실행 결과

```
## 학습 피쳐 데이터셋 info()
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 7352 entries, 0 to 7351
Columns: 561 entries, tBodyAcc-mean()-X to angle(Z,gravityMean)
dtypes: float64(561)
memory usage: 31.5 MB
```

■ 결과 해석

학습 데이터는 **7,352건 × 561피쳐**입니다. 모든 피쳐가 **float64** 이고 **결측치가 없어** 별도의 전처리 없이 바로 학습할 수 있습니다. 센서 데이터는 이미 **-1~1 사이로 정규화**되어 있습니다.

```
print(y_train['action'].value_counts())
```

▶ 실행 결과

```
action
6    1407
5    1374
4    1286
1    1226
2    1073
3     986
Name: count, dtype: int64
```

■ 결과 해석

6개 동작이 **986~1407건으로 비교적 고르게** 분포합니다. 심한 불균형이 없으므로 **정확도를 평가 지표로 사용해도** 무리가 없습니다. 동작 코드는 1=WALKING, 2=WALKING_UPSTAIRS, 3=WALKING_DOWNSTAIRS, 4=SITTING, 5=STANDING, 6=LAYING입니다.

4.4 기본 결정 트리 성능

```
from sklearn.tree import DecisionTreeClassifier
from sklearn.metrics import accuracy_score

# 예제 반복 시 마다 동일한 예측 결과 도출을 위해 random_state 설정
dt_clf = DecisionTreeClassifier(random_state=156)
dt_clf.fit(X_train, y_train)
pred = dt_clf.predict(X_test)
accuracy = accuracy_score(y_test, pred)
print('결정 트리 예측 정확도: {:.4f}'.format(accuracy))

# DecisionTreeClassifier의 하이퍼 파라미터 추출
print('DecisionTreeClassifier 기본 하이퍼 파라미터:\n', dt_clf.get_params())
```

줄	코드	상세 설명
5~7	dt_clf.fit / predict	모든 하이퍼 파라미터를 기본값으로 두고 학습·예측합니다.
12	dt_clf.get_params()	BaseEstimator를 상속했기 때문에 사용 가능한 메서드로, 현재 설정된 모든 하이퍼 파라미터 를 딕셔너리로 반환합니다. 튜닝 전 기본값을 확인하는 데 유용합니다.

▶ 실행 결과

```
결정 트리 예측 정확도: 0.8548
DecisionTreeClassifier 기본 하이퍼 파라미터:
{'ccp_alpha': 0.0, 'class_weight': None, 'criterion': 'gini', 'max_depth': None,
 'max_features': None, 'max_leaf_nodes': None, 'min_impurity_decrease': 0.0,
 'min_samples_leaf': 1, 'min_samples_split': 2, 'min_weight_fraction_leaf': 0.0,
 'monotonic_cst': None, 'random_state': 156, 'splitter': 'best'}
```

■ 결과 해석

기본 설정으로 **85.48%**의 정확도를 얻었습니다. 주목할 것은 **max_depth=None**이라는 점입니다. 즉 **깊이 제한 없이** 트리가 확장되었으며, 앞 절에서 본 것처럼 **과적합 가능성**이 있습니다.

⚠ 버전 차이

사이킷런 1.4부터 `monotonic_cst` (단조 제약) 파라미터가 추가되었습니다. 구버전 교재의 출력에는 이 항목이 없습니다. 동작에는 영향이 없습니다.

4.5 GridSearchCV로 max_depth 튜닝

```
from sklearn.model_selection import GridSearchCV

params = {
    'max_depth' : [ 6, 8 ,10, 12, 16 ,20, 24]
}

grid_cv = GridSearchCV(dt_clf, param_grid=params, scoring='accuracy', cv=5, verbose=1 )
grid_cv.fit(X_train , y_train)
print('GridSearchCV 최고 평균 정확도 수치:{0:.4f}'.format(grid_cv.best_score_))
print('GridSearchCV 최적 하이퍼 파라미터:', grid_cv.best_params_)
```

줄	코드	상세 설명
3~5	params = {'max_depth': [6,8,10,12,16,20,24]}	트리 깊이를 7가지 로 탐색합니다. 6부터 24까지 넓은 범위를 잡아 깊이와 성능의 관계 를 파악합니다.
7	GridSearchCV(dt_clf, param_grid=params, scoring='accuracy', cv=5, verbose=1)	7개 조합 × 5폴드 = 35번의 학습 이 수행됩니다. <code>verbose=1</code> 은 진행 상황을 출력합니다.
9~10	best_score_ / best_params_	최고 교차 검증 평균 정확도 와 그때의 파라미터를 출력합니다.

▶ 실행 결과

```
Fitting 5 folds for each of 7 candidates, totalling 35 fits
GridSearchCV 최고 평균 정확도 수치:0.8512
GridSearchCV 최적 하이퍼 파라미터: {'max_depth': 10}
```

```
# GridSearchCV객체의 cv_results_ 속성을 DataFrame으로 생성.
cv_results_df = pd.DataFrame(grid_cv.cv_results_)

# max_depth 파라미터 값과 그때의 테스트(Evaluation)셋, 학습 데이터 셋의 정확도 수치 추출
cv_results_df[['param_max_depth', 'mean_test_score']]
```

▶ 실행 결과

```
param_max_depth  mean_test_score
0                6      0.850791
1                8      0.851069
2               10      0.851209
3               12      0.844271
4               16      0.850255
5               20      0.850120
6               24      0.849168
```


■ 결과 해석 — 깊을수록 좋은 것이 아니다

max_depth=10에서 0.851209로 최고를 기록한 뒤, 그 이상에서는 **정확도가 오히려 떨어지거나 정체**합니다. 특히 12에서 0.8443으로 눈에 띄게 하락합니다.

이는 **트리가 깊어질수록 학습 데이터에 과적합**되어 검증 데이터에서의 성능이 나빠진다는 것을 실증적으로 보여 줍니다. 앞 절의 결정 경계 그림에서 본 현상이 실제 데이터에서 그대로 나타난 것입니다.

4.6 테스트 세트에서의 depth별 성능

```
max_depths = [ 6, 8 ,10, 12, 16 ,20, 24]
# max_depth 값을 변화 시키면서 그때마다 학습과 테스트 셋에서의 예측 성능 측정
for depth in max_depths:
    dt_clf = DecisionTreeClassifier(max_depth=depth, min_samples_split=16, random_state=156)
    dt_clf.fit(X_train , y_train)
    pred = dt_clf.predict(X_test)
    accuracy = accuracy_score(y_test , pred)
    print('max_depth = {0} 정확도: {1:.4f}'.format(depth , accuracy))
```

줄	코드	상세 설명
3~4	for depth in max_depths: DecisionTreeClassifier(max_depth=depth, min_samples_split=16, ...)	이번에는 교차 검증이 아니라 별도의 테스트 세트 에서 성능을 측정합니다. <code>min_samples_split=16</code> 도 함께 적용해 추가 제약을 줍니다.
7	accuracy_score(y_test, pred)	학습에 전혀 사용되지 않은 테스트 세트 에서의 정확도입니다.

▶ 실행 결과

```
max_depth = 6 정확도: 0.8551
max_depth = 8 정확도: 0.8717
max_depth = 10 정확도: 0.8599
max_depth = 12 정확도: 0.8571
max_depth = 16 정확도: 0.8599
max_depth = 20 정확도: 0.8565
max_depth = 24 정확도: 0.8565
```

■ 결과 해석 — 교차 검증과 테스트 세트의 최적값이 다르다

기준	최적 max_depth	정확도
교차 검증(학습 데이터)	10	0.8512
테스트 세트	8	0.8717

두 기준의 최적값이 다릅니다. 이는 **이상한 일이 아니라 자연스러운 현상**입니다. 데이터 분할 방식이 다르기 때문입니다. 다만 **모델 선택은 반드시 교차 검증 결과로** 해야 하며, 테스트 세트 점수를 보고 파라미터를 고르면 **테스트 세트에 과적합**됩니다.

공통적으로 확인되는 사실은 **깊이 8~10 부근이 최적**이고 그 이상은 **무의미**하다는 것입니다.

4.7 max_depth와 min_samples_split 동시 튜닝

```
params = {
    'max_depth' : [ 8 , 12, 16 ,20],
    'min_samples_split' : [16, 24],
}
```

```
grid_cv = GridSearchCV(dt_clf, param_grid=params, scoring='accuracy', cv=5, verbose=1 )
grid_cv.fit(X_train , y_train)
print('GridSearchCV 최고 평균 정확도 수치: {0:.4f}'.format(grid_cv.best_score_))
print('GridSearchCV 최적 하이퍼 파라미터:', grid_cv.best_params_)
```

줄	코드	상세 설명
1~4	params = {'max_depth':[8,12,16,20], 'min_samples_split':[16,24]}	두 개의 파라미터를 동시에 탐색합니다. 조합은 $4 \times 2 = 8$ 가지이며, 5폴드이므로 40번 학습합니다. max_depth 범위를 앞 결과에 맞춰 8 이상으로 좁혔습니다.

▶ 실행 결과

```
Fitting 5 folds for each of 8 candidates, totalling 40 fits
GridSearchCV 최고 평균 정확도 수치: 0.8549
GridSearchCV 최적 하이퍼 파라미터: {'max_depth': 8, 'min_samples_split': 16}
```

```
# 전체 조합별 교차 검증 평균 정확도
max_depth=8 min_samples_split=16 mean=0.854879 ← 최적
max_depth=8 min_samples_split=24 mean=0.851342
max_depth=12 min_samples_split=16 mean=0.845768
max_depth=12 min_samples_split=24 mean=0.846175
max_depth=16 min_samples_split=16 mean=0.847127
max_depth=16 min_samples_split=24 mean=0.849439
max_depth=20 min_samples_split=16 mean=0.848624
max_depth=20 min_samples_split=24 mean=0.849167
```

■ 결과 해석

max_depth=8, min_samples_split=16에서 0.8549로, 단일 파라미터 튜닝(0.8512)보다 향상되었습니다. 두 규제를 함께 적용했을 때 과적합이 더 효과적으로 억제된 것입니다.

모든 조합에서 max_depth=8이 가장 우수하며, 12 이상에서는 성능이 떨어집니다.

4.8 최종 모델 성능과 피쳐 중요도

```
best_df_clf = grid_cv.best_estimator_
pred1 = best_df_clf.predict(X_test)
accuracy = accuracy_score(y_test , pred1)
print('결정 트리 예측 정확도:{0:.4f}'.format(accuracy))
```

줄	코드	상세 설명
1	best_df_clf = grid_cv.best_estimator_	refit=True (기본값) 덕분에 최적 파라미터로 이미 재학습된 모델이 반환됩니다.
2~4	predict / accuracy_score	테스트 세트에서 최종 성능을 측정합니다.

▶ 실행 결과

```
결정 트리 예측 정확도:0.8717
```

■ 결과 해석 — 튜닝의 효과

모델	테스트 정확도
기본 결정 트리 (제약 없음)	0.8548
튜닝된 결정 트리 (max_depth=8, min_samples_split=16)	0.8717

튜닝만으로 약 **1.7%p** 향상되었습니다. 주목할 점은 성능을 높인 방법이 모델을 더 복잡하게 만든 것이 아니라 오히려 단순하게 제약한 것이라는 사실입니다.

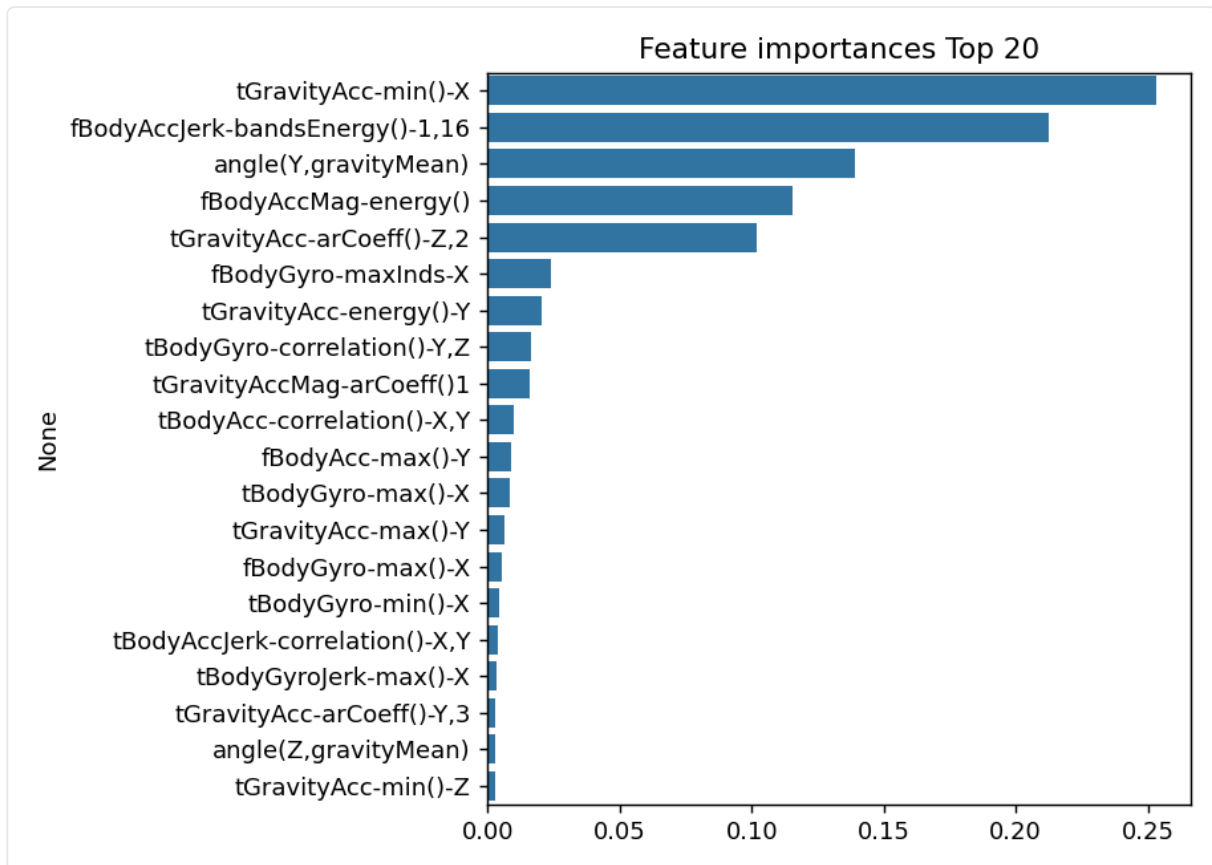
```
import seaborn as sns

ftr_importances_values = best_df_clf.feature_importances_
# Top 중요도로 정렬을 쉽게 하고, 시본(Seaborn)의 막대그래프로 쉽게 표현하기 위해 Series변환
ftr_importances = pd.Series(ftr_importances_values, index=X_train.columns )
# 중요도값 순으로 Series를 정렬
ftr_top20 = ftr_importances.sort_values(ascending=False)[:20]
plt.figure(figsize=(8,6))
plt.title('Feature importances Top 20')
sns.barplot(x=ftr_top20 , y = ftr_top20.index)
plt.show()
```

줄	코드	상세 설명
5	ftr_importances = pd.Series(ftr_importances_values, index=X_train.columns)	중요도 배열을 Series 로 변환하면서 인덱스에 피처명 을 부여합니다. 561개 숫자만으로는 어느 피처인지 알 수 없기 때문입니다.
7	ftr_importances.sort_values(ascending=False) [:20]	내림차순 정렬 후 상위 20개만 잘라냅니다. 561개를 모두 그리면 읽을 수 없습니다.
10	sns.barplot(x=ftr_top20, y=ftr_top20.index)	x에 값, y에 인덱스(피처명) 를 넣어 가로 막대 그래프를 그립니다.

▶ 실행 결과

```
tGravityAcc-min()-X      0.253354
fBodyAccJerk-bandsEnergy()-1,16  0.212584
angle(Y,gravityMean)      0.139001
fBodyAccMag-energy()       0.115478
tGravityAcc-arCoeff()-Z,2  0.101885
fBodyGyro-maxInds-X       0.023971
tGravityAcc-energy()-Y     0.020470
tBodyGyro-correlation()-Y,Z 0.016513
tGravityAccMag-arCoeff()1  0.015741
tBodyAcc-correlation()-X,Y 0.009972
... (이하 생략)
```



결정 트리 피쳐 중요도 상위 20개

■ 결과 해석

- 상위 5개 피쳐가 전체 중요도의 약 82%를 차지합니다. 561개 피쳐 중 극소수만 실제로 사용된 것입니다.
- 1위인 `tGravityAcc-min()-X` 는 중력 가속도의 X축 최솟값입니다. 눕기·서기·앉기처럼 몸의 방향이 달라지는 동작을 구분하는 데 결정적입니다.
- `angle(Y,gravityMean)` 도 중력 방향과의 각도로, 같은 맥락입니다.
- 결정 트리는 한 번에 하나의 피쳐로만 분할하므로 소수 피쳐에 중요도가 집중되는 경향이 있습니다. 다음 장의 랜덤 포레스트와 비교해 보면 이 차이가 뚜렷합니다.

5. 핵심 요약 및 머신러닝 활용 포인트

5.1 핵심 API 요약표

API / 속성	역할	주의 사항
<code>DecisionTreeClassifier()</code>	결정 트리 분류기	회귀는 <code>DecisionTreeRegressor</code>
<code>criterion</code>	분할 품질 지표	기본 'gini', 대안 'entropy'
<code>max_depth</code>	트리 최대 깊이	기본 None(무제한) → 과적합 위험
<code>min_samples_split</code>	분할 최소 샘플 수	기본 2. 높이면 규제 강화
<code>min_samples_leaf</code>	리프 최소 샘플 수	기본 1. 높이면 규제 강화

<code>feature_importances_</code>	피쳐별 중요도	합이 1. 학습 후 생성
<code>get_params()</code>	하이퍼 파라미터 조회	<code>BaseEstimator</code> 상속으로 제공
<code>export_graphviz()</code>	트리를 .dot 파일로 저장	Graphviz 별도 설치 필요
<code>make_classification()</code>	분류용 가상 데이터 생성	시각화 실습에 유용

5.2 실습 결과 종합

단계	설정	정확도
기본 결정 트리	제약 없음(max_depth=None)	0.8548
GridSearchCV 1차	max_depth만 탐색 → 10	0.8512 (교차 검증)
GridSearchCV 2차	max_depth=8, min_samples_split=16	0.8549 (교차 검증)
최종 테스트 세트	max_depth=8, min_samples_split=16	0.8717

5.3 머신러닝 활용 포인트

- **결정 트리는 반드시 규제한다** : 기본값은 무제한 확장이라 과적합이 거의 확실합니다. `max_depth` 는 최소한 지정하세요.
- **단순한 모델이 더 좋을 수 있다** : 본 실습에서 깊이 8이 깊이 24보다 우수했습니다. 복잡도를 높이는 것이 항상 답은 아닙니다.
- **시각화로 이해하기** : `export_graphviz` 와 결정 경계 그림은 **모델이 왜 그렇게 예측했는지** 설명할 때 강력한 도구입니다. 이런 **설명 가능성**이 결정 트리의 최대 장점입니다.
- **피쳐 중요도로 피쳐 선택** : 중요도 0인 피쳐는 제거해도 무방합니다. 561개 중 상위 20개만으로도 상당한 성능을 낼 수 있습니다.
- **스케일링이 불필요** : 트리는 값의 크기가 아니라 **분할 기준**만 사용하므로 정규화·표준화가 필요 없습니다.
- **단일 트리의 한계** : 85~87%가 단일 결정 트리의 한계입니다. 다음 장의 **랜덤 포레스트(4.4)**는 같은 데이터에서 **92% 이상**을 달성합니다. 여러 트리를 결합하는 **앙상블**이 답입니다.

⚠ 안정화 버전 참고 (scikit-learn 1.7.x 기준)

- `get_params( )` 출력에 `monotonic_cst` 가 추가되었습니다(1.4부터). 구버전 교재 출력과 다릅니다.
- `get_new_feature_name_df( )` 의 `lambda x: x[0]` 방식은 pandas 2.x에서 **위치 기반 접근 경고**가 발생할 수 있습니다. `x['column_name']` 처럼 **이름으로 접근**하는 것을 권장합니다.
- `read_csv` 의 `sep='\s+'` 는 pandas 2.x에서 **정규식 경고**가 뜰 수 있습니다. `sep=r'\s+'` 처럼 **raw 문자열**로 쓰면 해결됩니다.
- Graphviz 시각화는 파이썬 패키지(`pip install graphviz`)와 시스템 **Graphviz 프로그램**을 모두 설치해야 동작합니다. 설치 없이 쓰려면 사이킷런의 `plot_tree( )` 를 사용할 수 있습니다.